Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which of the following are valid ways to initialize an array in Java?

Seleccione una o más de una:

- a. int[] arr = (1, 2, 3);
- b. int[] arr = {1, 2, 3}; ✓
- c. int[] arr = new int[] {1, 2, 3}; ✓
- d. int[] arr = new int[3]; ✓

Justificación: Son válidos el inicializador abreviado {1, 2, 3}, new int[]{1, 2, 3} y new int[3], que deja los valores por defecto en 0. Los paréntesis (1, 2, 3) no son sintaxis de arreglo en Java y no compilan.

Fuente: Java.pdf #178

Las respuestas correctas son: int[] arr = {1, 2, 3};, int[] arr = new int[] {1, 2, 3};, int[] arr = new int[3];

Mi justificación
Para inicializar un array se necesita indicar que es un array con [] después del tipo de dato o del nombre de la variable, y si se construye ya el array con sus valores, usar llaves {} para llenarlo. El único que no cumple es la opción a) ya que usa paréntesis en lugar de llaves, los demás están bien.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué imprime este código de matriz?

```
int array[][] = {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
int result = 0;
for(int i = 0; i < array.length; i++) {
    result += array[i][i];
}
System.out.println(result);
```

Seleccione una:

- a. 0
- b. 9
- c. 18
- d. 15 ✓

Justificación: El bucle suma solo la diagonal principal array[i][i]: 1 + 5 + 9 = 15. 9 sería tomar solo el último elemento de la diagonal, y 0 supondría que el bucle no se ejecuta.

Fuente: Quiz APX #6

La respuesta correcta es: 15

Mi justificación
El arreglo array es 2D, así que recorrer array.length solo recorrerá la longitud de los arreglos primarios los cuales son 3, y para cada uno sumará a result el elemento en posición i, entonces solo sumará 1, 5 y 9, dando como resultado 15.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class NestedForLoopTest {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (i == j) {
                    System.out.print(matrix[i][j] + " ");
                }
            }
        }
    }
}
```

Seleccione una:

- a. 147
- b. 123456789
- c. 159 ✓
- d. 159246837

Justificación: Solo se imprimen las posiciones con i == j, es decir, la diagonal principal: [0][0]=1, [1][1]=5, [2][2]=9. La opción 147 correspondería a la primera columna (j == 0).

Fuente: Java.pdf #153 · también aparece en: Cuestionario APX V3 #28

La respuesta correcta es: 159

Mi justificación
Aquí ocurre un caso similar al problema 2 solo que aquí se imprimen los elementos donde i == j, que es lo mismo que imprimir [i][i], entonces se imprime solo 159.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Test {
    public static void main(String[] args){
        int[][] array = {{0},{0,1},{0,2,4},{0,3,6,9},{0,4,8,12,16}};
        System.out.println(array[4][1]);
        System.out.println(array[1][4]);
    }
}
```

Seleccione una:

- a. 4 Null
- b. Null 4.
- c. 4 An ArrayIndexOutOfBoundsException is thrown at run time. ✓
- d. An IllegalArgumentException is thrown at run time

Justificación: array[4] es {0,4,8,12,16}, así que array[4][1] vale 4. array[1] es {0,1}, de longitud 2, así que array[1][4] lanza ArrayIndexOutOfBoundsException. Los arreglos de primitivos nunca contienen null, por eso fallan las opciones con 'Null'.

Fuente: Java DE #23 · también aparece en: Java Sesiones #52

La respuesta correcta es: 4 An ArrayIndexOutOfBoundsException is thrown at run time.

Mi justificación
El arreglo array es 2D, contando con 5 arreglos primarios, cada uno con 1, 2, 3, 4 y 5 elementos cada uno respectivamente. Entonces. Imprimir array[4][1] sería el segundo elemento del 5to arreglo primario, entonces se imprime 4. Luego, al querer imprimir array[1][4] ocurre un OutOfBoundsException ya que el primer arreglo primerio solo tiene dos elementos ósea que [4] es un desbordamiento.

Pregunta 5

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public static void main(String[] args) {
    int[][] array2D = {{0,1,2}, {3,4,5,6}};
    System.out.print(array2D[0].length +"");
    System.out.print(array2D[1].getClass().isArray() +"");
    System.out.println(array2D[0][1]);
}
```

Seleccione una:

- a. 3false3
- b. 3true1
- c. 3false1 ⊗
- d. 2false1
- e. 2false3
- f. 2true3
- g. 2true1

Justificación: array2D[0] tiene 3 elementos. array2D[1] es un int[], así que getClass().isArray() da true. array2D[0][1] vale 1. Resultado: "3true1". El distractor 3false1 supone que un elemento de una matriz no es un arreglo.

Fuente: Java D #8 · también aparece en: Java Sesiones #26

La respuesta correcta es: 3true1

Mi justificación
El primer print pide la longitud del primer arreglo primario, el cual es 3. El segundo print pregunta si el 2do arreglo primario es un arreglo, pero getClass() no era necesario, así que da false. El último print regresa el segundo elemento del primer arreglo primario así que imprime 1.

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Qué imprime este código?

```
String[] fruts = {"Orange", "Apple", "Banana"};
Arrays.sort(fruts);
System.out.println(Arrays.binarySearch(fruts, "Banana"));
```

Seleccione una:

- a. 1 ✓
- b. 0
- c. 2
- d. -1

Justificación: Arrays.sort ordena lexicográficamente: [Apple, Banana, Orange]. binarySearch devuelve el índice de "Banana", que es 1. El distractor 2 es la posición original de "Banana" antes de ordenar. Si no se encontrara, binarySearch devolvería -(punto de inserción) - 1.

Fuente: Quiz APX #7

La respuesta correcta es: 1

Mi justificación
Al ordenar el arreglo fruts, queda Apple, Banana, Orange. El binarySeach encuentra Banana en la posición i = 1 así que imprime 1.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Que imprime en pantalla el código anterior?

```
import java.util.Arrays;
public class Test {
    public static void main(String[] args) {
        String[] strings = { "2", "3", "1", "4" };
        Arrays.sort(strings, 1, 4);
        System.out.println(Arrays.toString(strings));
    }
}
```

Seleccione una:

- a. [2, 3, 1, 4]
- b. [2, 1, 3, 4] ✓
- c. [1, 2, 3, 4]
- d. [2, 4, 1, 3]
- e. [4, 3, 2, 1]

Justificación: Arrays.sort(a, 1, 4) ordena solo los índices 1 a 3, porque el fin es exclusivo: {"3","1","4"} pasa a {"1","3","4"}. El índice 0 ('2') no se toca. El distractor [1, 2, 3, 4] supone que se ordena todo el arreglo.

Fuente: Simulación APX V1 #27 · también aparece en: Simulación APX V4 #23, Java.pdf #63

La respuesta correcta es: [2, 1, 3, 4]

Mi justificación
El método sort ordenará los elementos con índices 1 a 4-1 = 3, es decir que ordenará 3, 1, 4 regresándolo como 1, 3, 4. El arreglo final queda como 2, 1, 3, 4.

Pregunta 8

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public static void main(String[]args){
    int[] array = { 1,2,3,4,5};
    System.arraycopy(array, 2, array, 1, 2);
    System.out.print(array[1]);
    System.out.print(array[4]);
}
```

Seleccione una:

- a. 14
- b. 15
- c. 34
- d. 24
- e. 35
- f. 25 ⊗

Justificación: arraycopy copia 2 elementos desde el índice 2 (el 3 y el 4) al índice 1, y el arreglo queda {1,3,4,4,5}. array[1] vale 3 y array[4] sigue valiendo 5. El distractor 34 supone que también se modificó la última posición.

Fuente: Java D #2 · también aparece en: Java Sesiones #31

La respuesta correcta es: 35

?

Mi justificación
No sabía cómo funciona arraycopy así que la respondí al azar.

Pregunta 9

Parcialmente correcta

Se puntúa 0,50 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

Which two are valid instantiations and initializations of a multidimensional array?

Seleccione una o más de una:

- a. int[][] array2D = new int[2][2]; array2D[0][0] = 1; array2D[0][1] = 2; array2D[1][0] = 3; array2D[1][1] = 4;
- b. int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; array3D[0][1] = array; array3D[1][0] = array; array3D[0][1] = array;
- c. int[][] array2D = {0, 1};
- d. int[][][] array3D = {{0, 1}, {2, 3}, {4, 5}};
- e. int[][] array2D = {{0, 1, 2, 4}{5, 6}};

Ha seleccionado demasiadas opciones.

Justificación: Son válidos int[][] array2D = new int[2][2] seguido de asignar enteros a cada celda, e int[][][] array3D = new int[2][2][2] seguido de asignar un int[] a cada array3D[i][j] (repetir [0][1] no es un error). Fallan {0, 1} para un int[][] (los elementos deben ser int[], no int), {{0,1},{2,3},{4,5}} para un int[][][] (le falta un nivel de anidamiento) y {{0,1,2,4}{5,6}}, al que le falta la coma entre los subarreglos.

Nota sobre el material original: En el formulario original las opciones estaban mal partidas: la declaración de cada bloque válido había quedado al final de la opción anterior (por ejemplo, «int[][] array2D = new int[2][2];» al final de la e). Se reconstruyó cada opción con su declaración, como en la pregunta OCA original; las respuestas correctas siguen siendo a y b.

Fuente: Java D #7

Las respuestas correctas son: int[][] array2D = new int[2][2]; array2D[0][0] = 1; array2D[0][1] = 2; array2D[1][0] = 3; array2D[1][1] = 4;, int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; array3D[0][1] = array; array3D[1][0] = array; array3D[0][1] = array;

?

Mi justificación
XXX c) no es válido ya que el arreglo está declarado como bidimensional y solo se construye como unidimensional, d) ya que el arreglo está declarado como tridimensional y solo se construye como bidimensional. Las demás son correctas.

RESULTADO = 6/9 = 6.7